

CARDIOIA - FASE 5

IR ALÉM 2 - Automação Inteligente com RPA, IA e Dados Híbridos

Relatório Técnico

Projeto	CardioIA
Fase	5 - Conectando Mundos: IA Multimodal em Aplicações
Atividade	Assistente Cardiológico Inteligente: Experiência do Paciente
Componente	IR ALÉM 2 - Automação Inteligente com RPA, IA e Dados Híbridos
Finalidade	Protótipo acadêmico / simulação

Documento preparado com base na implementação existente, nos testes executados e nas evidências de execução fornecidas para o projeto.

1. Contexto e objetivo

O IR ALÉM 2 amplia o CardiolA por meio de um fluxo automatizado que combina automação robótica de processos (RPA), análise automatizada e uso conjunto de armazenamento relacional e não relacional. O protótipo simula a atuação de um robô sobre dados clínicos estruturados, identifica situações que demandam atenção e registra os resultados de forma rastreável.

Objetivos da implementação

- Consultar dados clínicos estruturados a partir de uma base relacional.
- Orquestrar o processamento por meio de uma rotina de automação em Python.
- Aplicar regras de análise automatizada para identificar situações de atenção.
- Registrar resultados, alertas e metadados em armazenamento documental.
- Preservar data e hora para permitir rastreabilidade das execuções.

2. Arquitetura implementada

Camada	Componente	Responsabilidade
Dados estruturados	SQLite / database.py	Fonte dos registros clínicos
Automação	backend/automation.py	Orquestra consulta, análise e persistência
Análise	backend/ai_analysis.py	Aplica regras e produz status/alertas
Dados não relacionais	backend/nosql.py	Grava e recupera eventos documentais
Persistência NoSQL	backend/cardioia_nosql.json	Armazena resultados e data/hora
Qualidade	tests/test_ai_analysis.py	Valida cenários normais e de atenção

Fluxo: Banco Relacional (SQLite) -> RPA / Automação Python -> análise automatizada -> identificação de alertas
-> Banco NoSQL -> registro de evento + data/hora.

3. Banco relacional

O projeto utiliza SQLite como banco relacional, com o arquivo **database/cardioia.db** e o módulo **database/database.py**. A base fornece os dados clínicos estruturados consumidos pela automação. Para a demonstração, os registros incluem identificador do paciente, pressão sistólica, pressão diastólica, frequência cardíaca, adesão ao tratamento e data/hora.

4. Automação RPA

O módulo **backend/automation.py** atua como orquestrador do processo. A rotina consulta os registros clínicos, percorre cada item, envia os dados para o módulo de análise, interpreta o resultado, monta um documento de evento e o encaminha ao armazenamento NoSQL.

- Consulta os dados clínicos no banco relacional.
- Processa cada registro individualmente.
- Executa a análise automatizada.
- Registra o status e os alertas encontrados.
- Persiste o resultado em formato documental.
- Exibe o andamento e o encerramento da execução.

5. Análise automatizada

O módulo **backend/ai_analysis.py** aplica regras determinísticas de análise para fins de simulação acadêmica. A implementação não representa diagnóstico médico real; o objetivo é demonstrar um mecanismo automatizado de identificação de situações de atenção.

Condição	Regra implementada	Alerta / resultado
Frequência cardíaca	> 100 bpm	Frequência cardíaca elevada
Frequência cardíaca	< 60 bpm	Frequência cardíaca baixa
Pressão arterial	Sistólica >= 140 ou diastólica >= 90	Pressão arterial elevada
Adesão	< 80%	Baixa adesão ao tratamento
Resultado final	Um ou mais alertas	ATENÇÃO
Resultado final	Nenhum alerta	NORMAL

6. Banco não relacional e rastreabilidade

O arquivo **backend/nosql.py** implementa uma camada documental baseada em JSON. A função **salvar_evento()** recupera os documentos existentes, acrescenta um novo evento, inclui automaticamente o campo **data_hora** e persiste o conjunto novamente. A função **buscar_eventos()** permite recuperar os eventos armazenados.

Os documentos de análise clínica registram, de forma estruturada, tipo do evento, origem, paciente, pressão, frequência cardíaca, adesão, status, alertas e data/hora. No fluxo demonstrado, a origem é identificada como **Cardiologia_RPA**.

7. Fluxo de dados híbridos

Etapa	Origem -> destino	Resultado
1	SQLite -> automation.py	Dados clínicos estruturados carregados
2	automation.py -> ai_analysis.py	Registro submetido à análise
3	ai_analysis.py -> automation.py	Status + lista de alertas
4	automation.py -> nosql.py	Evento documental estruturado
5	nosql.py -> cardioia_nosql.json	Persistência + data/hora

8. Testes automatizados

O arquivo **tests/test_ai_analysis.py** contém cinco testes automatizados:

- **test_paciente_normal** - valida um cenário sem alertas.
- **test_frequencia_cardiaca_elevada** - valida a detecção de frequência acima do limite adotado.
- **test_pressao_arterial_elevada** - valida a detecção de pressão elevada.
- **test_baixa_adesao** - valida a identificação de baixa adesão.
- **test_multiplos_alertas** - valida a coexistência de três alertas no mesmo registro.

Na execução registrada, o conjunto de testes terminou com **5 passed**.

9. Evidências de execução

As evidências abaixo documentam a execução real do protótipo e complementam a descrição técnica. Os exemplos foram preservados como registros representativos do funcionamento.

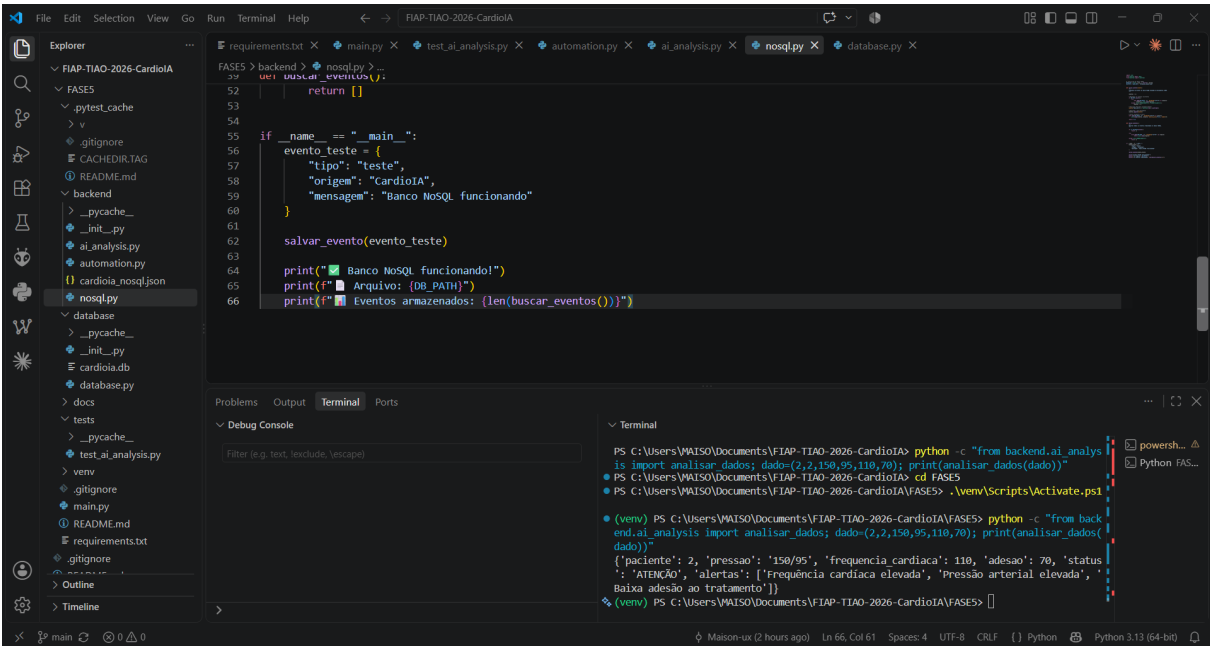


Figura 1 - Estrutura local da FASE5 no VS Code, com os módulos de backend, database, tests, ambiente virtual e arquivo NoSQL.

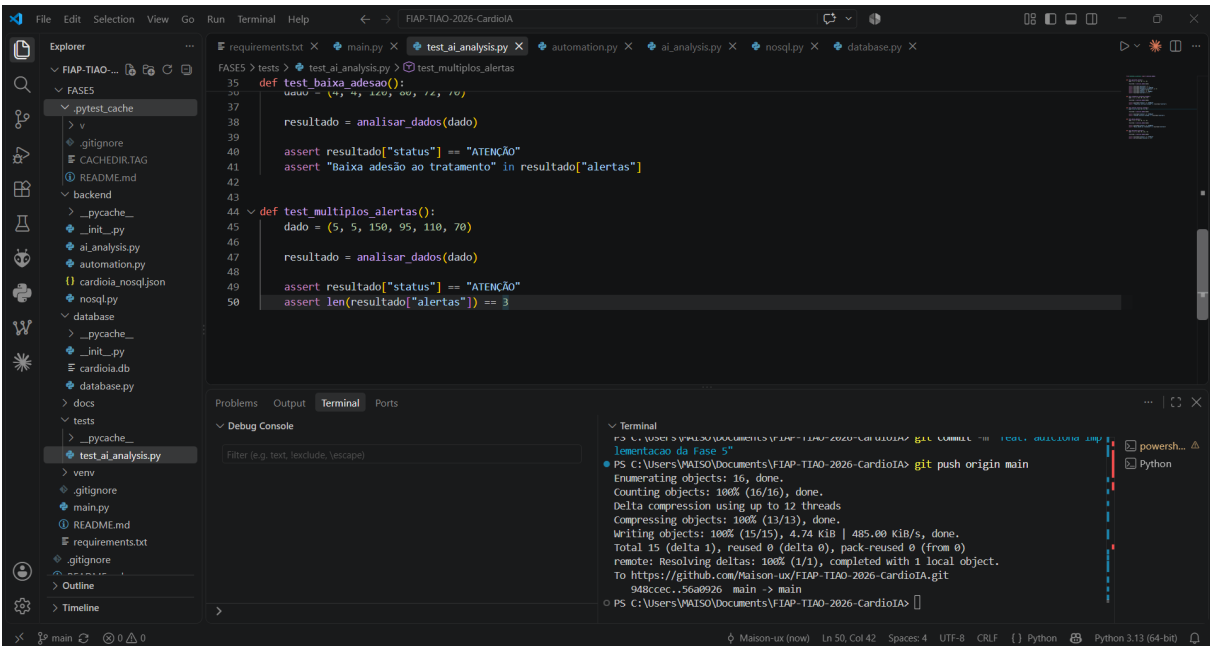


Figura 2 - Execução dos testes automatizados do módulo de análise; o terminal registra o push anterior e o arquivo de testes está visível no VS Code.

9.1. Cenários demonstrados

Cenário	Paciente	PA	FC	Adesão	Status	Alertas
NORMAL	1	120/80	72 bpm	95%	NORMAL	Nenhum
ATENÇÃO	2	150/95	110 bpm	70%	ATENÇÃO	3 alertas

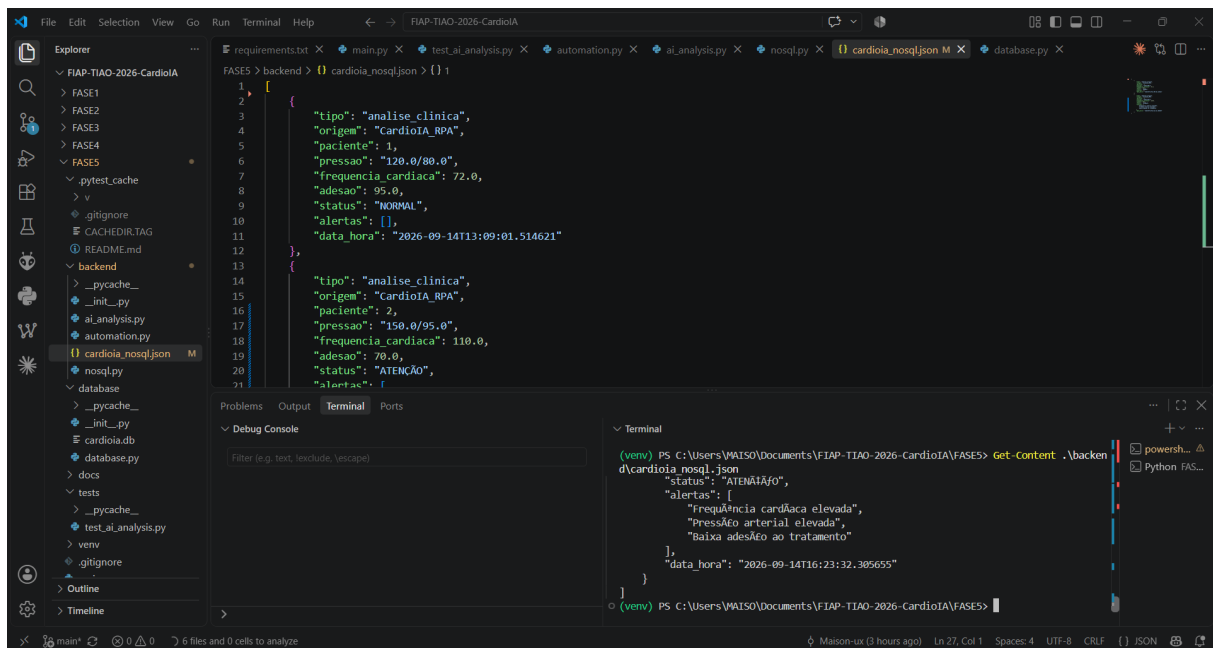


Figura 3 - Conteúdo do cardioia_nosql.json após a limpeza, mantendo registros de análise clínica, status, alertas e data_hora.

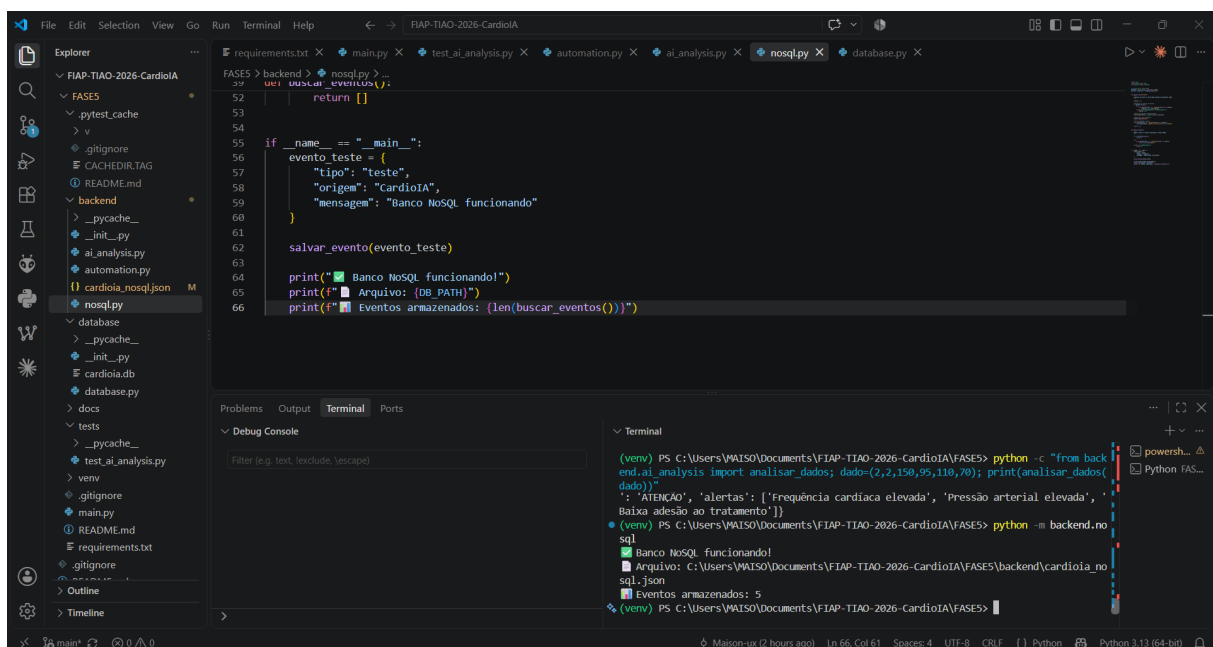


Figura 5 - Execução do módulo NoSQL, confirmando o armazenamento documental e a localização do arquivo cardioia_nosql.json.

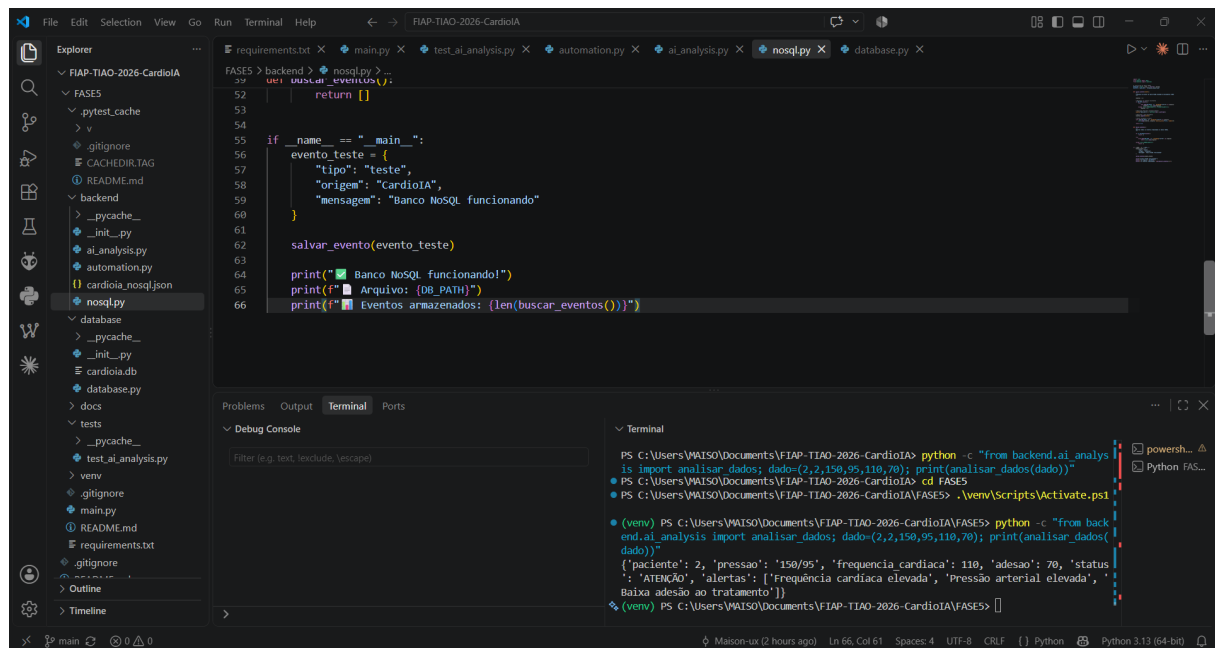


Figura 6 - Execução do cenário ATENÇÃO, com paciente 2 e os três alertas gerados pelo módulo de análise.

10. Resultados

- Banco relacional SQLite utilizado como fonte de dados clínicos estruturados.
- Fluxo de automação RPA implementado em Python.
- Análise automatizada integrada ao fluxo.
- Identificação de situações de atenção e geração de múltiplos alertas.
- Persistência dos resultados em armazenamento não relacional baseado em documentos JSON.
- Registro temporal por meio do campo `data_hora`.
- Cinco testes automatizados executados com sucesso (5 passed).

11. Rastreabilidade

A rastreabilidade é obtida por meio da combinação de origem do evento (**CardioIA_RPA**), identificação do paciente, dados avaliados, status, lista de alertas e data/hora. Esse conjunto permite relacionar o resultado produzido pela automação ao registro documental que foi persistido.

12. Conclusão

O IR ALÉM 2 demonstrou, em um protótipo acadêmico funcional, a integração prática entre **RPA + IA + banco relacional + banco NoSQL + rastreabilidade**. O fluxo automatiza a consulta dos registros, executa a análise, identifica situações de atenção e persiste os resultados em formato documental com registro temporal.

A implementação atende ao propósito de demonstrar automação e integração de dados híbridos em um contexto de saúde simulado, sem caracterizar o sistema como ferramenta de diagnóstico médico real.

13. Evidências visuais e publicação

As capturas de tela a seguir foram incorporadas ao relatório para comprovar a execução local, a análise, a persistência documental e a publicação do artefato.

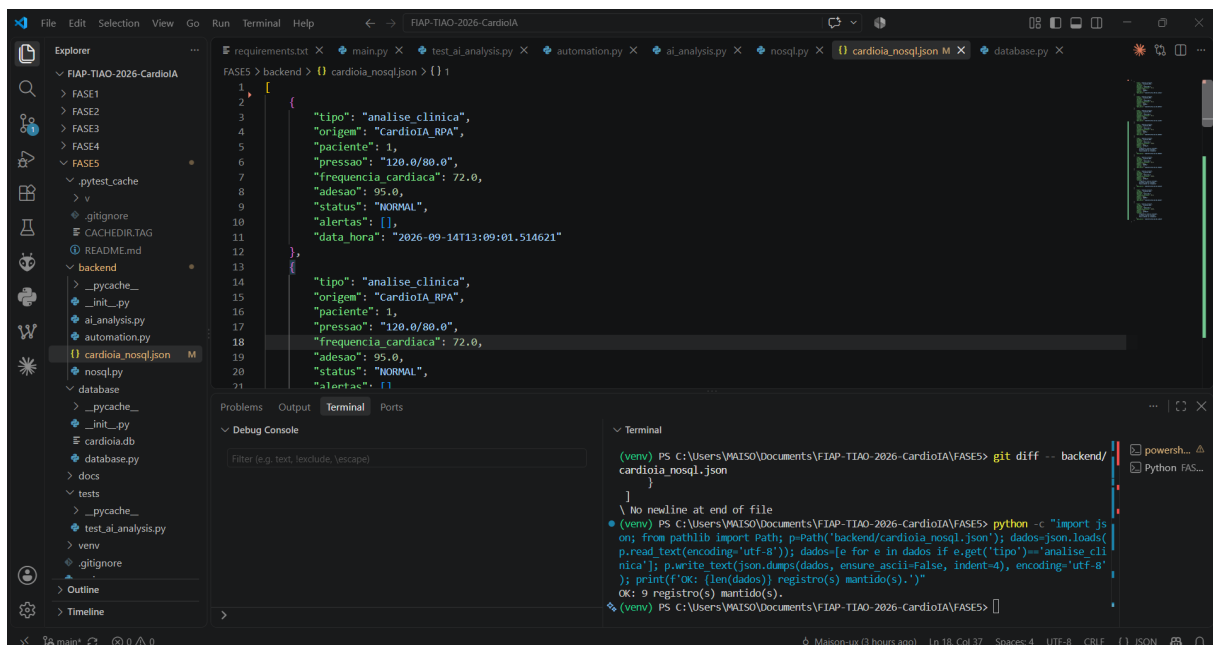


Figura 7 - Conferência local do JSON e do fluxo de persistência, com registros de análise clínica no artefato.

A Figura 7 complementa a evidência do conteúdo persistido no armazenamento documental. O conjunto de figuras deve ser lido em conjunto com os resultados descritos na seção 9.

14. Arquivos principais da solução

Arquivo	Função
backend/automation.py	Orquestra o fluxo RPA
backend/ai_analysis.py	Executa a análise automatizada
backend/nosql.py	Persiste e consulta documentos JSON
backend/cardioia_nosql.json	Armazenamento documental de evidências
database/database.py	Acesso ao banco SQLite
database/cardioia.db	Banco relacional
tests/test_ai_analysis.py	Testes automatizados
main.py	Entrada principal da Fase 5

Documento técnico - CardioIA Fase 5 / IR ALÉM 2